

# molchemist

## Molecule rendering from Molfile, SDF, and SMILES

v0.1.5      2026-08-22

MIT AND BSD-3-Clause AND Apache-2.0 AND (Apache-2.0 WITH LLVM-exception)

Render chemical structures from Molfile, SDF, and SMILES data.

Eito Yoneyama

`molchemist` renders chemical structures from Molfile / SDF data. It can also parse SMILES strings by generating a 2D layout first. The package turns molecular graphs into an `alchemist` drawing program and renders the final figure with `CeTZ`.

The package is aimed at Typst documents that need compact molecule figures, publication-oriented skeletal formulae, and light annotation without leaving Typst.

## Table of Contents

|                                              |           |
|----------------------------------------------|-----------|
| <b>Getting Started .....</b>                 | <b>2</b>  |
| <b>Compatibility and Test Coverage .....</b> | <b>3</b>  |
| <b>Example Data .....</b>                    | <b>4</b>  |
| <b>Input and Semantic Fidelity .....</b>     | <b>5</b>  |
| <b>Rendering Modes .....</b>                 | <b>18</b> |
| <b>Styling .....</b>                         | <b>19</b> |
| <b>Annotations .....</b>                     | <b>20</b> |
| <b>Dump Mode .....</b>                       | <b>27</b> |
| <b>Command-Line Export .....</b>             | <b>28</b> |
| <b>Publication Guidance .....</b>            | <b>29</b> |
| <b>SMILES Notes .....</b>                    | <b>30</b> |
| <b>API Reference .....</b>                   | <b>32</b> |
| <b>Limitations .....</b>                     | <b>39</b> |
| <b>License and Dependencies .....</b>        | <b>40</b> |
| <b>Index .....</b>                           | <b>41</b> |



## Part II

# Compatibility and Test Coverage

| Surface                                                         | Minimum   | Continuously tested                        |
|-----------------------------------------------------------------|-----------|--------------------------------------------|
| Typst package with <code>str</code> or <code>bytes</code> input | 0.14.0    | 0.14.0, 0.14.1, 0.14.2, 0.15.0, and 0.15.1 |
| Typst <code>path(...)</code> input                              | 0.15.0    | 0.15.0 and 0.15.1                          |
| <code>molchemist-cli</code> build                               | Rust 1.86 | Rust 1.86 on Ubuntu, macOS, and Windows    |

For each listed Typst version, CI compiles the rendering fixtures on Ubuntu and checks that package dump output is byte-for-byte identical to CLI output. Rust parser, formatter, CLI, and WASM-host tests additionally run on macOS and Windows. The matrix covers the published package entrypoint and CLI-generated source, not the separate third-party toolchain used to typeset this manual.

Compatibility with Typst 0.14.0 and 0.14.1 is tested for users who cannot upgrade, but those releases contain an [upstream WebAssembly runtime security issue](#). Prefer Typst 0.14.2 or later for production documents.

## Part III

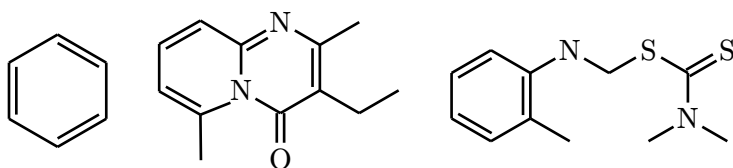
# Example Data

The SDF examples in this manual use real PubChem records included in the repository test data.

- PubChem CID 241: benzene.
- PubChem CID 93406: 3-ethyl-2,6-dimethylpyrido[1,2-a]pyrimidin-4-one.
- PubChem SID 93298: a deposited DTP/NCI substance record associated with CID 235403.

```
#let benzene = read("Structure2D_COMPOUND_CID_241.sdf")
#let fused = read("Structure2D_COMPOUND_CID_93406.sdf")
#let deposited = read("DepositedStructure_SUBSTANCE_SID_93298_Version_3.sdf")

#grid(
  columns: 3,
  gutter: 7mm,
  align: center + horizon,
  render-mol(benzene, skeletal: true, config: (atom-sep: 1.55em)),
  render-mol(fused, skeletal: true, config: (atom-sep: 1.55em)),
  render-mol(deposited, skeletal: true, config: (atom-sep: 1.55em)),
)
```



## Part IV

# Input and Semantic Fidelity

The examples in this chapter focus on information that is easy to lose when a molecular file is reduced to a generic graph. Each example shows both the Typst source and the resulting drawing so that input selection, metadata, bond meaning, and stereochemistry can be checked independently.

## IV.1 SDF Versions and Record Selection

`#render-mol` detects V2000 and V3000 independently for every selected SDF record. The `(record)` option is one-based and defaults to 1; it does not depend on the format of preceding records. This makes mixed-version SDF collections usable without preprocessing.

```
#let records = read("structures.sdf")

#grid(
  columns: 2,
  gutter: 10mm,
  align: center + top,
  [
    *V2000 · record 1*
    #v(2mm)
    #render-mol(records, record: 1)
  ],
  [
    *V3000 · record 2*
    #v(2mm)
    #render-mol(records, record: 2, abbreviate: true)
  ],
)
```

V2000 · record 1

V3000 · record 2

C

$^{15}\text{N}^{+\bullet}:7-\text{O}^-$

The second synthetic record also demonstrates V3000 charge, isotope, radical, and atom-map fields. Bond configuration is covered separately in the stereochemistry examples below. An out-of-range record number, an empty structure, malformed CTAB data, or non-finite coordinates raises an explicit error instead of silently drawing the wrong record.

## IV.2 Semantic Inspection and Fidelity Policy

The `#inspect-mol` function exposes a versioned semantic dictionary before depiction. It contains the record header, stable source atom and bond IDs, enhanced stereo groups, SGroups and their attachment points, link nodes, collections, ordered SDF properties, the exact selected record, and diagnostics for parsed features the renderer cannot yet depict. V3000 source IDs retain their

original CTAB values; V2000 source IDs use one-based source positions. Unlike a dictionary, the property array preserves duplicate names and multiline values.

```
#let record = inspect-mol(read("structures.sdf"), record: 2)
#table(
  columns: 2,
  [Schema], [#record.schemaVersion],
  [Format], [#record.format],
  [First source ID], [#record.atoms.first().sourceId],
  [Properties], [#record.properties.len()],
  [Diagnostics], [#record.diagnostics.len()],
)
```

|                 |               |
|-----------------|---------------|
| Schema          | 1             |
| Format          | molfile-v3000 |
| First source ID | 1             |
| Properties      | 0             |
| Diagnostics     | 0             |

Labelled, unexpanded multi-atom superatoms are contracted in every fidelity mode. SGroup SAP entries, variable-attachment bonds, link nodes, and atom/bond HILITE collections are preserved in the same semantic record; features without a standard or implemented depiction remain explicit diagnostics. Use `render-mol(data, fidelity: "strict")` to reject those gaps, including reaction-center flags and arbitrary user-defined collections. The default "ignore" keeps existing document behavior. The command-line interface defaults to `--fidelity warn`; `molchemist inspect` writes the complete semantic record as JSON.

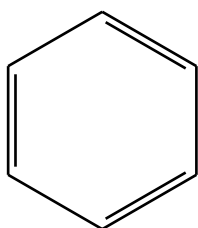
## IV.3 Coordinate Preservation and Layout Recovery

Usable source coordinates are preserved, including their relative orientation. Automatic layout is requested only when the XY geometry is unusable—for example, when every bonded atom is collapsed onto one point or a nominally 3D record has no meaningful XY projection. The recovered drawing still uses the source bond orders, wedge direction, atom metadata, and record ordering.

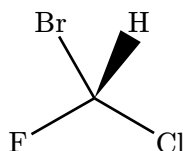
```
#let supplied = read("benzene-2d.sdf")
#let collapsed = read("collapsed-layout.sdf")

#grid(
  columns: 2,
  gutter: 12mm,
  align: center + top,
  [
    *Preserved 2D coordinates*
    #v(2mm)
    #render-mol(supplied, skeletal: true)
  ],
  [
    *Recovered collapsed coordinates*
    #v(2mm)
    #render-mol(collapsed, skeletal: true)
  ],
)
```

Preserved 2D coordinates



Recovered collapsed coordinates



The fallback is deliberately conservative: crowded but numerically valid coordinates are not redrawn merely because their full-mode labels overlap. Change `(atom-sep)` or use abbreviated/skeletal mode when the geometry is valid but the typography is dense.

## IV.4 Atom Metadata and Explicit Labels

Bracket-atom metadata remains structured through the SMILES parser, WASM boundary, formatter, and Typst renderer. Isotope numbers and formal charges use their conventional upper positions, hydrogen counts use a lower position, and atom classes remain visible as an inline `:n` suffix. Invisible balancing attachments keep the primary atom symbols aligned when differently scripted labels appear in one disconnected structure.

```
#grid(
  columns: 3,
  gutter: 8mm,
  align: center + top,
  [
    *Atom class*
    #v(2mm)
    #render-smiles("[CH3:1]O", abbreviate: true)
  ],
  [
    *Isotope + class*
    #v(2mm)
    #render-smiles("[13CH3:7]C", abbreviate: true)
  ],
  [
    *Charge + H count*
    #v(2mm)
    #render-smiles("[NH4+].[Cl-]", skeletal: true)
  ],
)
```

---

| Atom class              | Isotope + class                                  | Charge + H count                             |
|-------------------------|--------------------------------------------------|----------------------------------------------|
| CH <sub>3</sub> :1 — OH | <sup>13</sup> CH <sub>3</sub> :7·CH <sub>3</sub> | NH <sub>4</sub> <sup>+</sup> Cl <sup>−</sup> |

In full mode, explicitly represented hydrogen atoms remain separate graph nodes. In abbreviated and skeletal modes, foldable hydrogens join their parent labels, except when folding would erase a stereochemical wedge/dash or change the meaning of a bridging hydrogen.

## IV.5 CTfile Queries, SGroups, Collections, and Attachments

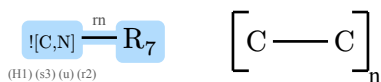
CTfile display information travels through the same semantic record and depiction AST. Atom lists use MDL-style glyphs such as `![C,N]`. Hydrogen count, substitution count, unsaturation, ring-bond count, and valence constraints use compact parenthetical annotations such as `(H1)`, `(s3)`, `(u)`, and `(r2)`; set config: `(ctfile: (query-details: true))` to show explicit labels such as implicit `H >= 1` instead.

MDL query `H0` means that no implicit hydrogen is allowed unless it is drawn explicitly; `Hn` means at least  $n$  implicit hydrogens in excess of hydrogens explicitly drawn. Consequently, V3000 `HCOUNT=-1` normalizes to `H0`, while source values `HCOUNT=1, 2, ..., 4` normalize directly to `H1, H2, ..., H4`; the source record remains available unchanged through `#inspect-mol`. R-group labels remain visible, and ring/chain bond topology uses the plain `rn / ch` annotations. Labelled, unexpanded multi-atom SUP SGroups contract to graph nodes; other known SGroup types use common brackets with adjacent labels. V3000 atom/bond MDLV30/HILITE collections render on

a background layer that follows glyph bounds. Ordered SDF properties remain available through inspection.

For substitution and ring-bond counts, the CTfile sentinel -2 is depicted as (s\*) / (r\*) (“as drawn”), and -1 as (s0) / (r0). Substitution values of six or more share the (s6) bucket; ring-bond values of four or more share (r4).

```
#let data = read("ctfile-fidelity.sdf")
#render-mol(
  data,
  fidelity: "strict",
  config: (
    ctfile: (
      highlight-paint: rgb("#74c0fc").transparentize(45%),
    ),
  ),
)
```

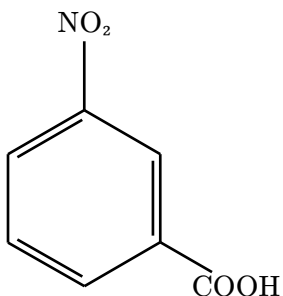


### IV.5.1 Multi-atom Superautom Contraction

An unexpanded SUP SGroup with more than one source atom contracts to one labelled graph node. Internal atoms and bonds remain available through `inspect-mol`, while the depiction removes internal bonds and reconnects every crossing bond to the contracted glyph. With one attachment atom, its source coordinate is retained; with several attachment atoms, their centroid defines the glyph position. Atom highlights on hidden members project to the glyph, crossing-bond highlights follow the reconnected bond, and an internal-bond-only highlight also resolves to the glyph.

The example below uses the real ACD/Labs V2000 N02 / C00H fixture distributed by RDKit. fidelity: "strict" succeeds because both labelled SGroups are contracted; N02 is typeset as NO<sub>2</sub> without changing the inspected CTfile label.

```
#let data = read("Sgroups_Abbreviations.mol", encoding: none)
#render-mol(data, skeletal: true, fidelity: "strict")
```

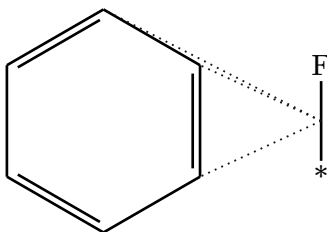


### IV.5.2 Attachment Points, Link Nodes, and Collections

SGroup M SAP and SAP=(...) entries remain structured as attachment atom, optional leaving atom, and connection ID; contracted multi-atom superatoms use those explicit attachment atoms to choose the glyph position. V2000 M AP0 and V3000 ATTCHPT are R-group-member attributes, not main-CTAB atom decorations. A value found on the main CTAB is therefore preserved but rejected by strict fidelity instead of being presented as a standard glyph.

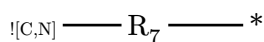
V2000 M LIN and V3000 LINKNODE entries render their minimum–maximum repeat range next to the link atom. A V3000 variable-attachment bond preserves ENDPTS and ATTACH: ANY alternatives use dotted branches, while ALL uses solid branches. Atom lists belong to atom types, and R-group numbers belong to RGROUPS; they are not COLLECTION kinds. MDLV30/HILITE has a defined renderer convention. Other internal or user-defined collections retain their full name, atom/bond/SGroup/3D/R-group member lists, generic members, and original logical entry through `#inspect-mol`, but no visual convention is invented for them.

```
#let data = read("ctfile-attachments-collections.sdf", encoding: none)
#align(center)[
  #render-mol(
    data, record: 1, skeletal: true, fidelity: "strict",
    config: (atom-sep: 3.8em),
  )
]
```



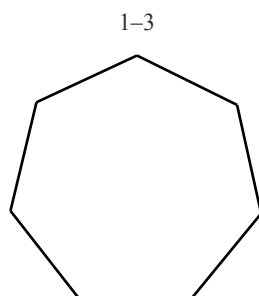
V3000 position variation: dotted branches are the ATTACH=ANY endpoint alternatives.

```
#let data = read("ctfile-attachments-collections.sdf", encoding: none)
#align(center)[
  #render-mol(
    data, record: 2, skeletal: true, fidelity: "strict",
    config: (atom-sep: 3.8em),
  )
]
```



Standard atom-list query and R-group 7 atom attributes; the terminal \* is an explicit wildcard atom.

```
#let data = read("Sgroups_Link_01.mol", encoding: none)
#align(center)[
  #render-mol(
    data, skeletal: true, fidelity: "strict",
    config: (atom-sep: 3.8em),
  )
]
```



Real V2000 M LIN fixture from RDKit; the link repeat range is 1-3.

```
#let data = read("ctfile-attachments-collections.sdf", encoding: none)
#let record = inspect-mol(data, record: 3)
#let collection = record.collections.first()
#table(
  columns: 2,
  [Name], [#raw(collection.name)],
  [Kind], [#collection.kind],
  [Atom IDs], [#collection.atomSourceIds.map(str).join(", ")],
  [Bond IDs], [#collection.bondSourceIds.map(str).join(", ")],
  [Diagnostic], [#record.diagnostics.first().code],
)
```

|            |                                      |
|------------|--------------------------------------|
| Name       | molchemist/SELECTION                 |
| Kind       | user-defined                         |
| Atom IDs   | 10                                   |
| Bond IDs   | 7                                    |
| Diagnostic | user-defined-collection-not-depicted |

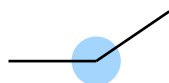
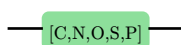
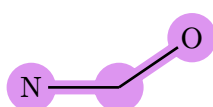
### IV.5.3 Highlight Depiction

V3000 MDLV30/HILITE collections keep atom and bond membership separate. A highlighted skeletal carbon uses a circle centered on the hidden vertex. A visible atom uses the same visual treatment, expanding to a rounded rectangle only when its rendered glyph bounds are wider than the configured radius. Bond-only highlights use a constant-width capsule with round ends.

Atom and bond subpaths share one non-zero compound fill, so connected selections read as one continuous region while disconnected selections remain visually separate.

```
#let data = read("highlight-shapes.sdf", encoding: none)
#let highlight-case(title, record, paint) = align(center)[
  *#title*
  #v(2mm)
  #render-mol(
    data, record: record, skeletal: true, fidelity: "strict",
    config: (ctfile: (
      highlight-paint: paint.transparentize(35%),
    )),
  )
]

#grid(
  columns: 2,
  gutter: 10mm,
  row-gutter: 8mm,
  align: top,
  highlight-case([Atom only · skeletal vertex], 1, rgb("#74c0fc")),
  highlight-case([Atom only · query glyph], 2, rgb("#51cf66")),
  highlight-case([Bond only · capsule], 3, rgb("#ffd43b")),
  highlight-case([Connected atom + bond], 4, rgb("#cc5de8")),
  highlight-case([Disconnected regions], 5, rgb("#ff922b")),
)
```

**Atom only · skeletal vertex****Atom only · query glyph****Bond only · capsule****Connected atom + bond****Disconnected regions**

The regression corpus also includes [RDKit/Bingo's real V3000 v3k.crash1.mol](#), which declares the same source ID in separate atom and bond HILITE collections. Semantic tests preserve those source memberships independently, and the SVG regression checks the compound-path topology for every case above.

## IV.6 Bond Semantics

V3000 bond orders 4 through 10 remain distinct: aromatic, single-or-double, single-or-aromatic, double-or-aromatic, any, coordination, and hydrogen bonds are not collapsed to ordinary single bonds. Query and aromatic bonds use dashed/dotted partial lines, any/either bonds use a wavy line, coordination bonds preserve their donor-to-acceptor direction with a filled arrowhead, and hydrogen bonds use a dotted line. A SMILES \$ bond is rendered separately as four parallel lines.

| V3000 order | Meaning            | Visual cue                             |
|-------------|--------------------|----------------------------------------|
| 4           | Aromatic           | Double line with a dashed partial line |
| 5           | Single or double   | Double line with a dotted partial line |
| 6           | Single or aromatic | Dashed single line                     |
| 7           | Double or aromatic | Dashed double line                     |
| 8           | Any                | Wavy line                              |
| 9           | Coordination       | Directed line with a filled arrowhead  |
| 10          | Hydrogen           | Dotted line                            |

```
#let extended = read("bond-semantics.sdf")

#grid(
  columns: 1,
  row-gutter: 5mm,
  align: center,
  [
    *V3000 extended bond orders*
    #v(2mm)
    #render-mol(
      extended,
      abbreviate: true,
      config: (atom-sep: 3.0em),
    )
  ],
  [
    *OpenSMILES quadruple bond*
    #v(2mm)
    #render-smiles("[Cr]${Cr}")
  ],
)
```

---

#### V3000 extended bond orders

----- N ..... O ----- S ===== P  F → Cl ..... Br  I

#### OpenSMILES quadruple bond

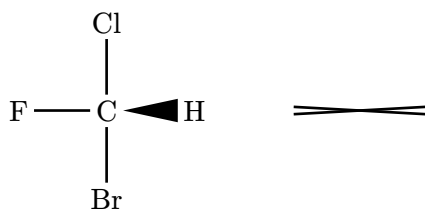
Cr  Cr

Long hydrogen bonds are excluded when the renderer computes a representative covalent bond length. This prevents a distant noncovalent contact from shrinking the covalent part of the structure.

## IV.7 SDF Stereochemical Metadata

For Molfile/SDF input, up/down single bonds remain wedge/dash bonds. Undefined double-bond geometry—V2000 stereo code 3 or V3000 double-bond CFG=2—uses a crossed double bond. Atom CFG parity and V3000 STEABS, STEREL, and STERAC collections are retained as annotations rather than being discarded.

```
#let stereo-data = read("stereochemistry.sdf")  
#render-mol(stereo-data, skeletal: true)
```



Stereo annotations: C CFG=1; OR7 (a0)

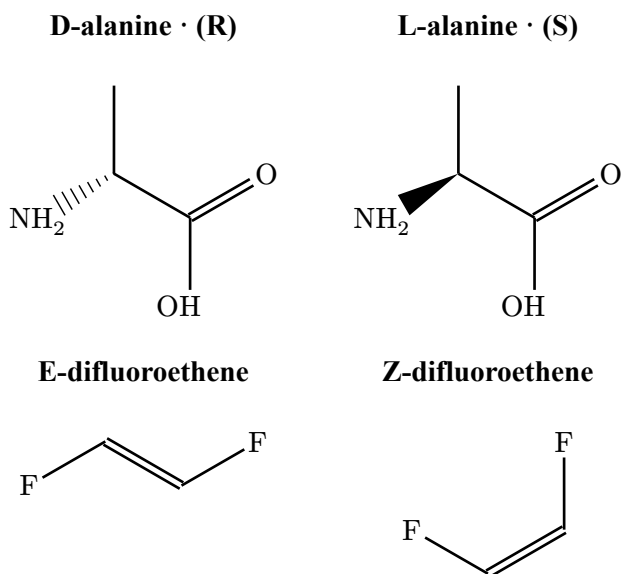
The explicit hydrogen above is intentionally kept visible because its wedge carries stereochemical information. The disconnected crossed double bond in the same record also demonstrates that component separation does not drop bond configuration.

## IV.8 SMILES Tetrahedral and Double-Bond Stereo

OpenSMILES @ and @@ are interpreted using local SMILES neighbor order, not by assigning a fixed wedge to one token. Branch order, bracket hydrogens, incoming atoms, and ring-closure token positions therefore contribute to the depicted configuration. Directional / and \ bonds are likewise resolved as a pair around the double bond.

```
#let stereo-example(title, source) = align(center)[
  *#title*
  #v(2mm)
  #render-smiles(source, skeletal: true)
]

#grid(
  columns: 2,
  gutter: 10mm,
  row-gutter: 6mm,
  align: top,
  stereo-example([D-alanine · (R)], "N[C@H](C)C(=O)O"),
  stereo-example([L-alanine · (S)], "N[C@@H](C)C(=O)O"),
  stereo-example([E-difluoroethene], "F/C=C/F"),
  stereo-example([Z-difluoroethene], "F/C=C\F"),
)
```



The implicit-hydrogen form N[C@@H](C)C(=O)O and the explicit-hydrogen form N[C@@]([H])(C)C(=O)O describe the same local configuration and are tested to produce the same absolute orientation.

## IV.9 Multi-component Structures

Disconnected Molfile/SDF graphs and dot-separated SMILES retain every component in source order. Components are separated by whitespace with no visible synthetic operator. Atom and bond indices remain global, so annotation anchors can target later components without renumbering them.

```
#grid(
  columns: 1,
  row-gutter: 5mm,
  align: center,
  [
    *Dot-separated salt*
    #v(2mm)
    #render-smiles("[Na+].[Cl-]", abbreviate: true)
  ],
  [
    *Visible isolated components*
    #v(2mm)
    #render-smiles("[H+].C.[Cl-]", skeletal: true)
  ],
)
```

---

### Dot-separated salt



### Visible isolated components



Isolated hydrogen and zero-heavy-neighbor carbon components remain visible in abbreviated and skeletal modes. Scripted labels are balanced around their primary symbols, so H, CH, and Cl align while the 4 in CH<sub>4</sub> remains correctly lowered.

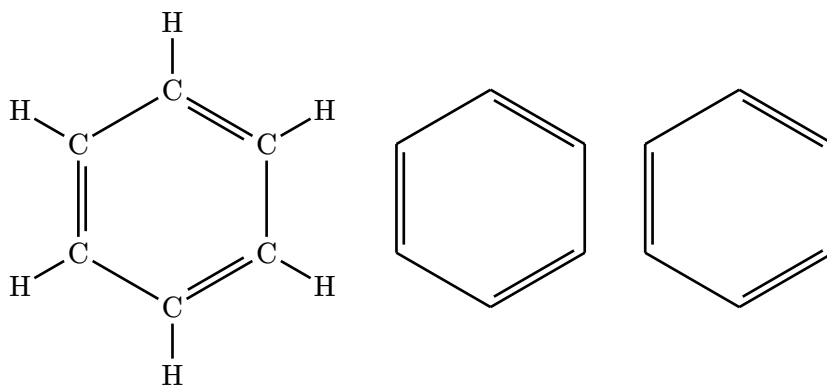
## Part V

# Rendering Modes

Both renderers support the same three modes. Full mode is useful for small molecules and debugging. Abbreviated mode folds common hydrogens and terminal carbons into labels. Skeletal mode is usually the best default for publication figures.

```
#let mol-data = read("Structure2D_COMPOUND_CID_241.sdf")

#grid(
  columns: 3,
  gutter: 8mm,
  align: center + horizon,
  render-mol(mol-data),
  render-mol(mol-data, abbreviate: true),
  render-mol(mol-data, skeletal: true),
)
```



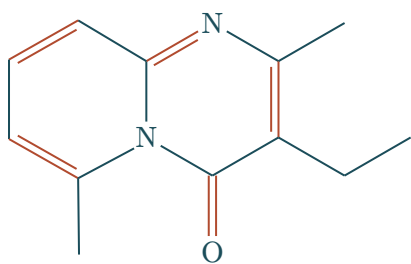
## Part VI

# Styling

Pass `<config>` for visual adjustments that should reach `alchemist`, such as atom spacing, fragment color, and bond styles.

```
#let mol-data = read("Structure2D_COMPOUND_CID_93406.sdf")

#render-mol(
  mol-data,
  skeletal: true,
  config: (
    atom-sep: 2.6em,
    fragment-color: rgb("#1b4d5a"),
    single: (stroke: 0.75pt + rgb("#1b4d5a")),
    double: (stroke: 0.75pt + rgb("#b14b2f")),
  ),
)
```



Molfile and SDF input normally contains usable coordinates; collapsed or numerically unstable coordinates receive an automatic 2D fallback layout. Routing-oriented `alchemist` settings do not reshape either graph; prefer `atom-sep`, `font`, and `stroke` settings for visual tuning.

## Part VII

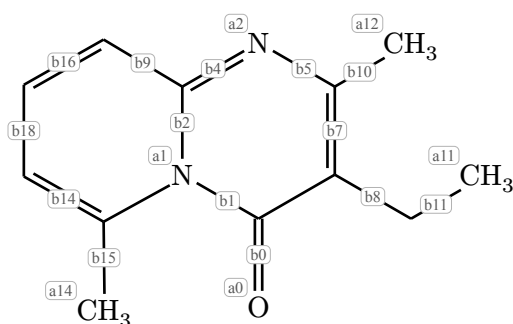
# Annotations

Annotations are drawn after the molecule. They are meant for sparse publication callouts, not for replacing a full figure editor.

## VII.1 Finding Atom and Bond Indices

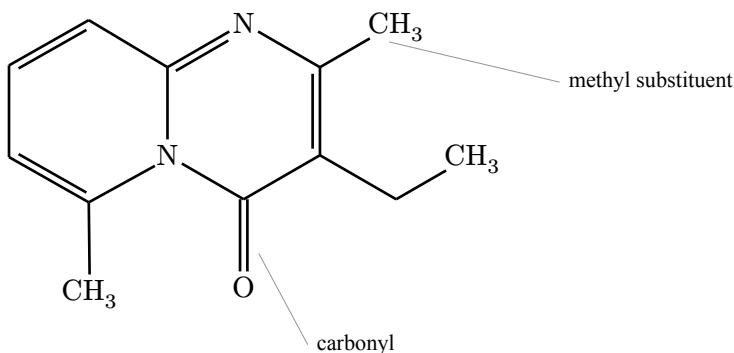
Enable `(show-indices)` while authoring. The visible labels are the indices used by `#atom-anchor` and `#bond-anchor`.

```
#let mol-data = read("Structure2D_COMPOUND_CID_93406.sdf")  
#render-mol(mol-data, abbreviate: true, show-indices: true)
```



```
#let mol-data = read("Structure2D_COMPOUND_CID_93406.sdf")

#render-mol(
  mol-data,
  abbreviate: true,
  annotations: (
    callout-annotation(
      bond-anchor(0),
      [carbonyl],
      label-at: (to: molecule-anchor(anchor: "south"), rel: (0.82, -0.48)),
      leader-start: (to: molecule-anchor(anchor: "south"), rel: (0.7, -0.48)),
      leader-end: (to: bond-anchor(0), rel: (0.2, -0.24)),
      leader: "curve",
      stroke: luma(58%) + 0.28pt,
      label-size: 0.76em,
      label-anchor: "west",
    ),
    callout-annotation(
      atom-anchor(12),
      [methyl substituent],
      label-at: (to: molecule-anchor(anchor: "east"), rel: (0.9, 1.04)),
      leader-start: (to: molecule-anchor(anchor: "east"), rel: (0.78, 1.04)),
      leader-end: (to: atom-anchor(12), rel: (0.28, -0.38)),
      leader: "curve",
      stroke: luma(58%) + 0.28pt,
      label-size: 0.76em,
      label-anchor: "west",
    ),
  ),
),
)
```

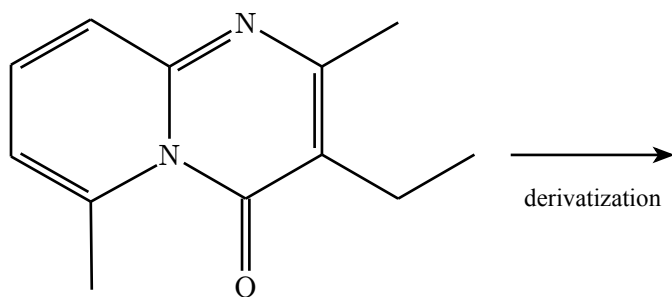


## VII.3 Arrows

Use `#arrow-annotation` for molecule-level process arrows or simple directional marks.

```
#let mol-data = read("Structure2D_COMPOUND_CID_93406.sdf")

#render-mol(
  mol-data,
  skeletal: true,
  annotations: arrow-annotation(
    (to: molecule-anchor(anchor: "east", rel: (0.48, 0)),
    (to: molecule-anchor(anchor: "east", rel: (2.65, 0)),
    label: [derivatization],
    label-offset: (0, -0.46),
    label-anchor: "north",
  ),
)
```

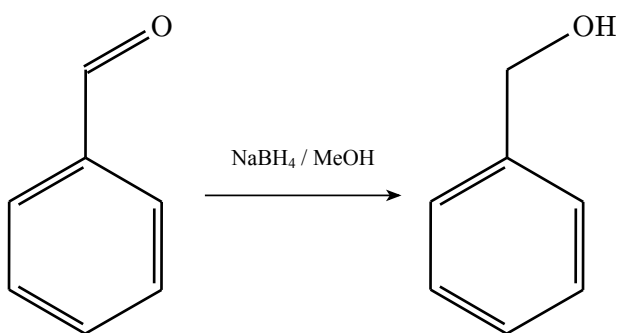


## VII.4 Reaction Schemes

For reactions, compose separate molecule renderings with normal Typst/CeTZ layout. This keeps reaction arrows independent from molecule annotations.

```
#let reaction-arrow = cetz.canvas({
  import cetz.draw: *
  line(
    (0.1, 0), (2.7, 0),
    stroke: 0.65pt + black,
    mark: (end: ">>", scale: 0.72, fill: black),
  )
  content((1.4, 0.4), text(size: 0.76em)[NaBH4 / MeOH], anchor: "south")
})

#grid(
  columns: (auto, 26mm, auto),
  column-gutter: 4mm,
  align: horizon + center,
  render-smiles("O=CC1=CC=CC=C1", abbreviate: true),
  reaction-arrow,
  render-smiles("OCC1=CC=CC=C1", abbreviate: true),
)
```

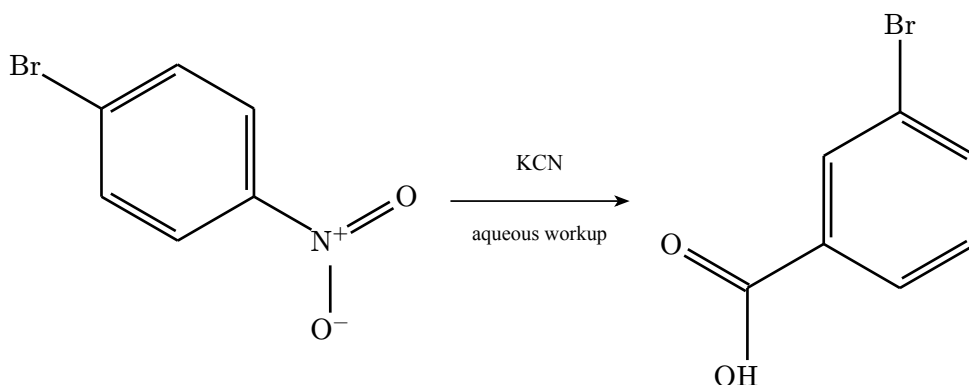


## VII.5 Schematic von Richter Transformation

The package does not yet parse reaction formats or infer reaction centers. Reaction figures therefore compose separately rendered molecules. The net conversion below is a schematic overview, not an exhaustive electron-pushing mechanism: cyanide addition and rearrangement are represented by the labelled arrow, and the product is shown after aqueous workup.

```
#let scheme-arrow(above, below) = cetz.canvas({
  import cetz.draw: *
  line(
    (0.08, 0), (2.45, 0),
    stroke: 0.65pt + black,
    mark: (end: ">", scale: 0.72, fill: black),
  )
  content((1.26, 0.4), text(size: 0.76em)[#above], anchor: "south")
  content((1.26, -0.34), text(size: 0.7em)[#below], anchor: "north")
})

#grid(
  columns: (auto, 24mm, auto),
  column-gutter: 4mm,
  align: horizon + center,
  render-smiles("O=[N+](O-)c1ccc(Br)cc1", skeletal: true),
  scheme-arrow([KCN], [aqueous workup]),
  render-smiles("O=C(O)c1ccc(Br)cc1", skeletal: true),
)
```



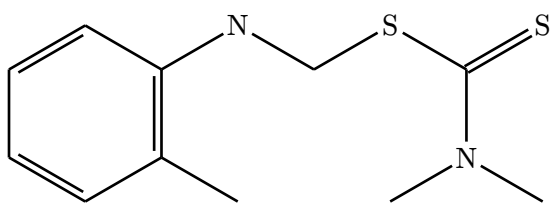
This transformation is discussed by M. Rosenblum, *The Mechanism of the von Richter Reaction*, J. Am. Chem. Soc. 82 (1960), 3796–3798 (DOI 10.1021/ja01499a090). A detailed mechanistic figure should cite a chosen mechanistic model and draw its individual intermediates explicitly.

## VII.6 Low-Level Labels

Use `#label-annotation` when no leader line is needed.

```
#let mol-data = read("DepositedStructure_SUBSTANCE_SID_93298_Version_3.sdf")

#render-mol(
  mol-data,
  skeletal: true,
  annotations: label-annotation(
    molecule-anchor(anchor: "south"),
    [PubChem SID 93298],
    offset: (0, -0.65),
    label-anchor: "north",
  ),
)
```



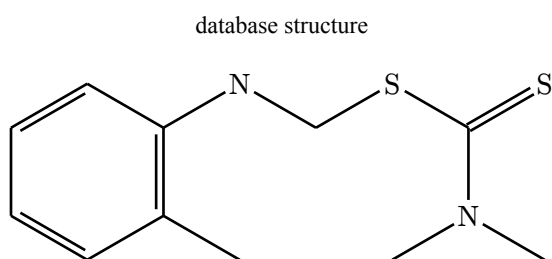
PubChem SID 93298

## VII.7 Custom CeTZ Overlays

For final figure polishing, `#cetz-annotation` exposes the generated molecule name, so you can draw directly against CeTZ anchors.

```
#let mol-data = read("DepositedStructure_SUBSTANCE_SID_93298_Version_3.sdf")

#render-mol(
  mol-data,
  skeletal: true,
  annotations: cetz-annotation(mol => {
    import cetz.draw: *
    content(
      (to: (name: mol, anchor: "north"), rel: (0, 0.54)),
      text(size: 0.82em)[database structure],
      anchor: "south",
    )
  }),
)
```



## Part VIII

# Dump Mode

With `(dump)`, `molchemist` returns generated `alchemist` source instead of a drawing. Use this when a figure needs manual surgery beyond the annotation API.

```
#let mol-data = read("Structure2D_COMPOUND_CID_241.sdf")
#render-mol(mol-data, skeletal: true, dump: true)

#let base-sep = 3em
#skeletalize({
  hook("a0")
  branch({
    double(absolute: -29.997863113888922deg, atom-sep: base-sep *
      1.2346644251497605, offset: "right", name: "b0")
    hook("a1")
    single(absolute: -90deg, atom-sep: base-sep * 1.2345846673836163,
      name: "b3")
    hook("a3")
    double(absolute: -150.0021368861111deg, atom-sep: base-sep *
      1.2346644251497605, offset: "right", name: "b7")
    hook("a5")
    single(absolute: 149.99927221917264deg, atom-sep: base-sep *
      1.2345575062221579, name: "b9")
    hook("a4")
    double(absolute: 90deg, atom-sep: base-sep * 1.2345846673836163,
      offset: "right", name: "b5")
    hook("a2")
    branch({
      single(absolute: 0deg, atom-sep: 0pt, stroke: none, name: "a2-
        links", links: (
          "a0": single(absolute: 30.000727780827372deg, atom-sep: base-sep *
            1.2345575062221579, name: "b1"),
        ))
    })
  })
})
```

## Part IX

# Command-Line Export

For scripts and editor workflows, install the `molchemist-cli` crate with `cargo install --locked molchemist-cli`. Its `molchemist dump` command accepts Molfile, SDF, or SMILES input and writes the same formatted source as `{dump}` to standard output. Add `--standalone` to include the current alchemist import and an auto-sized page, or `--output figure.typ` to write directly to a file.

The normal output is an editable Alchemist program rather than an image or diagnostic transcript. For example, the command and its complete standard output are:

```
$ molchemist dump --smiles 'CC(=O)O' --mode skeletal
#let base-sep = 3em
#skeletize({
  hook("a0")
  single(absolute: 29.79036703670196deg, atom-sep: base-sep * 1, name:
    "b0")
  hook("a1")
  branch({
    double(absolute: 89.79373607661383deg, atom-sep: base-sep *
      1.000062954206203, name: "b1")
    fragment("O", name: "a2")
  })
  single(absolute: -30.20116835518715deg, atom-sep: base-sep *
    0.9999817671902098, name: "b2")
  fragment("OH", name: "a3")
})
```

Redirect this snippet when it will be imported or edited inside an existing document. Use `{record}` for a multi-record SDF, or request a complete compilable document with `{standalone}`:

```
$ molchemist dump molecule.sdf > molecule.typ
$ molchemist dump structures.sdf --record 2 --mode skeletal > record-2.typ
$ molchemist dump --smiles 'CC(=O)O' --mode skeletal \
  --standalone --output acetic-acid.typ
$ typst compile acetic-acid.typ
```

Input may also be piped from another program—for example, `printf '%s\n' 'c1ccccc1' | molchemist dump --format smiles > benzene.typ`. Diagnostics are written to standard error and generated source exclusively to standard output, so redirection does not mix warnings or errors into a `.typ` file.

## Part X

# Publication Guidance

For paper figures, start with `<skeletal>` for hydrocarbon-heavy structures and `<abbreviate>` when heteroatom hydrogens or terminal groups should remain explicit. Keep full mode for small structures and debugging.

Keep annotations sparse. In most cases, a thin unboxed `#callout-annotation` is better than a boxed label or an arrowhead. If a leader line looks like a chemical bond, move the label with `<label-at>` or stop the leader outside the structure with `<leader-end>`.

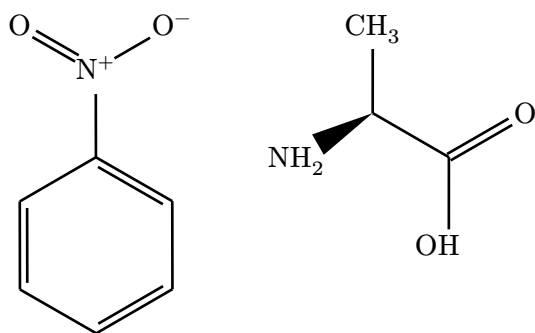
Dense structures can still overlap in full mode, especially after SMILES implicit hydrogens are expanded. Prefer abbreviated or skeletal mode when a molecule is meant for a publication figure.

## Part XI

# SMILES Notes

SMILES support is a parse-and-layout pipeline. The parser accepts common SMILES notation, aromatic rings, charges, tetrahedral @ / @@ centers, and / / \ double-bond geometry. It rejects malformed branch, dot, bond, bracket-property, charge, isotope, atom-class, directional-bond, and aromatic notation instead of normalizing it silently. Atom classes from 0 through 9999 are accepted, and aromatic systems must satisfy Hückel's rule and admit a valence-compatible Kekulé assignment. Tetrahedral centers retain OpenSMILES local neighbor order, including bracketed hydrogens and ring-closure token positions.

```
#grid(  
  columns: 2,  
  gutter: 10mm,  
  align: top + center,  
  render-smiles("O=[N+](O-)[c1ccccc1]", abbreviate: true),  
  render-smiles("N[C@@H](C)C(=O)O", abbreviate: true),  
)
```



Extended OpenSMILES chirality classes are rendered geometrically when their topology permits an unambiguous projection. @AL uses terminal wedge/dash bonds, @SP uses its U/4/Z ligand path, and @TB / @OH combine the specified ligand winding with solid and hashed viewing-axis bonds. Invalid or cyclic topologies that cannot be rearranged safely retain their original chirality tag as a fallback annotation.

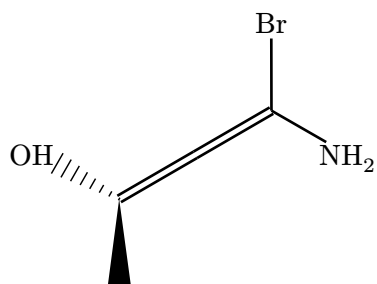
```

#let chiral-example(title, source) = align(center)[
  *#title*
  #v(2mm)
  #render-smiles(source, skeletal: true)
]

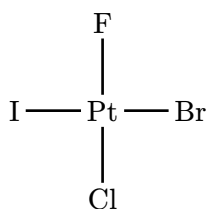
#grid(
  columns: 2,
  gutter: 8mm,
  row-gutter: 7mm,
  align: top,
  chiral-example([Allene · `@AL1`], "NC(Br)=[C@AL1]=C(O)C"),
  chiral-example([Square planar · `@SP2`], "[Pt@SP2](F)(Cl)(Br)I"),
  chiral-example([Trigonal bipyramidal · `@TB5`], "[As@TB5](F)(Cl)(Br)(N)S"),
  chiral-example([Octahedral · `@OH5`], "[Co@OH5](F)(Cl)(Br)(I)(N)S"),
)

```

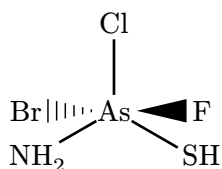
Allene · @AL1



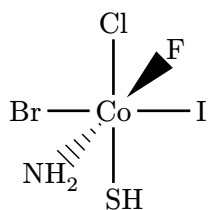
Square planar · @SP2



Trigonal bipyramidal · @TB5



Octahedral · @OH5



# Part XII

## API Reference

### XII.1 Rendering Functions

```
#render-mol(  
  {data},  
  {record}: 1,  
  {abbreviate}: false,  
  {skeletal}: false,  
  {dump}: false,  
  {config}: {},  
  {annotations}: none,  
  {show-indices}: false,  
  {fidelity}: "ignore"  
) → content
```

Render a molecule from raw Molfile or SDF text.

Usable input coordinates are preserved. Collapsed or numerically unstable coordinates receive a generated 2D layout while retaining source metadata and bond stereochemistry.

Argument

{data}

str | bytes | path

Raw .mol or .sdf data. Typst 0.15.0 and later may pass path(...) directly; older versions should pass read(...) output.

Argument

{record}: 1

int

One-based record number for multi-record SDF input.

Argument

{abbreviate}: false

bool

Enables abbreviated rendering.

Argument

{skeletal}: false

bool

Enables skeletal rendering. This overrides {abbreviate}.

Argument

{dump}: false

bool

Returns generated alchemist source code instead of rendering the molecule.

Argument

`(config): ( : )`

dictionary

Visual configuration passed to `alchemist`. The optional `ctfile` subdictionary is consumed by `molchemist` and accepts `highlight-paint`, `highlight-radius`, `highlight-thickness`, `highlight-label-padding`, `query-details`, `sgroup-stroke`, `sgroup-label-size`, `link-node-size`, and `variable-attachment-stroke`.

`highlight-radius` is the radius used for an unlabeled skeletal atom, `highlight-thickness` is the full width of a highlighted bond capsule, and `highlight-label-padding` expands the rendered glyph bounds before drawing a rounded rectangle. All three values scale with `(atom-sep)`.

Argument

`(annotations): none`

annotation | array | none

Optional overlay annotation or array of annotations.

Argument

`(show-indices): false`

bool | str

Debug overlay for annotation authoring. Use `true`, `"all"`, `"atoms"`, or `"bonds"`.

Argument

`(fidelity): "ignore"`

str

Use `"strict"` to reject known remaining features that are preserved by inspection but not fully depicted. `"ignore"` keeps the normal rendering behavior.

**#inspect-mol**(`(data)`, `(record): 1`) → dictionary

Parse one Molfile/SDF record into the versioned semantic IR without reducing it to drawing commands.

Argument

`(data)`

str | bytes | path

Raw `.mol` or `.sdf` data. Typst 0.15.0 and later may pass `path(...)` directly; older versions should pass `read(...)` output.

Argument

`(record): 1`

int

One-based record number for multi-record SDF input.

The result includes `schemaVersion`, `format`, `header fields`, `atoms`, `bonds`, `stereoGroups`, `sgroups`, `linkNodes`, `collections`, `ordered properties`, `diagnostics`, and `rawRecord`. Atom and bond entries contain both a zero-based depiction index and a stable `sourceId`; atoms may contain `normalized query`, `rgroupLabels`, and `attachmentPoints`, bonds may contain `endpointSourceIds` and `attachmentMode`, and `SGroups` contain both `normalized kind` and `original typeCode` plus structured `SAP attachmentPoints`. A collection contains its complete

name, normalized kind, atom/bond/SGroup/3D/R-group member IDs, generic members, and exact rawEntry.

```
#render-smiles(  
  {smiles},  
  {abbreviate}: false,  
  {skeletal}: false,  
  {dump}: false,  
  {config}: (:),  
  {annotations}: none,  
  {show-indices}: false  
) → content
```

Parse a SMILES string, generate 2D coordinates, and render the resulting molecule.

|                  |     |
|------------------|-----|
| Argument         |     |
| {smiles}         | str |
| A SMILES string. |     |

|                                                        |      |
|--------------------------------------------------------|------|
| Argument                                               |      |
| {abbreviate}: false                                    | bool |
| Enables abbreviated rendering after layout generation. |      |

|                                                          |      |
|----------------------------------------------------------|------|
| Argument                                                 |      |
| {skeletal}: false                                        | bool |
| Enables skeletal rendering. This overrides {abbreviate}. |      |

|                                                                            |      |
|----------------------------------------------------------------------------|------|
| Argument                                                                   |      |
| {dump}: false                                                              | bool |
| Returns generated alchemist source code instead of rendering the molecule. |      |

|                                           |            |
|-------------------------------------------|------------|
| Argument                                  |            |
| {config}: (:)                             | dictionary |
| Visual configuration passed to alchemist. |            |

|                                                      |                           |
|------------------------------------------------------|---------------------------|
| Argument                                             |                           |
| {annotations}: none                                  | annotation   array   none |
| Optional overlay annotation or array of annotations. |                           |

|                                                                               |            |
|-------------------------------------------------------------------------------|------------|
| Argument                                                                      |            |
| {show-indices}: false                                                         | bool   str |
| Debug overlay for annotation authoring. Use true, "all", "atoms", or "bonds". |            |

## XII.2 Anchor Helpers

**#atom-anchor**(**{index}**, **{anchor}**: "mid") → **anchor**

Select an atom anchor for annotation placement.

Argument —  
**{index}** int  
Atom index shown by **{show-indices}**.

Argument —  
**{anchor}**: "mid" str  
CeTZ anchor on the atom object, such as "north", "east", or "mid".

**#bond-anchor**(**{index}**, **{anchor}**: "50%") → **anchor**

Select a bond anchor for annotation placement.

Argument —  
**{index}** int  
Bond index shown by **{show-indices}**.

Argument —  
**{anchor}**: "50%" str  
CeTZ anchor on the bond object.

**#molecule-anchor**(**{anchor}**: "center") → **anchor**

Select an anchor on the whole rendered molecule.

Argument —  
**{anchor}**: "center" str  
CeTZ group anchor such as "center", "north", "east", "south", or "west".

**#atom-ref**(**{index}**) → **str**

Return the internal atom anchor name as a string.

**#bond-ref**(**{index}**) → **str**

Return the internal bond anchor name as a string.

## XII.3 Annotation Builders

```
#callout-annotation(
  {at},
  {label},
  {anchor}: "mid",
  {side}: "north-east",
  {label-at}: auto,
  {label-offset}: auto,
  {target-offset}: auto,
  {target-gap}: 0.2,
  {label-anchor}: auto,
  {leader}: "curve",
  {leader-start}: auto,
  {leader-end}: auto,
  {leader-points}: (),
  {leader-start-offset}: (0, 0),
  {leader-end-offset}: (0, 0),
  {mark}: none,
  {stroke}: 0.35pt + luma(40%),
  {label-size}: 0.82em,
  {label-gap}: 0.14,
  {label-inset}: 2.2pt,
  {label-radius}: 3pt,
  {label-fill}: luma(100%),
  {label-stroke}: 0.35pt + luma(70%),
  {boxed}: false,
  {name}: none
) → annotation
```

Build an external label with a leader line.

|                              |                |
|------------------------------|----------------|
| Argument                     |                |
| {at}                         | <b>anchor</b>  |
| Target anchor.               |                |
| Argument                     |                |
| {label}                      | <b>content</b> |
| Label content.               |                |
| Argument                     |                |
| {anchor}: "mid"              | <b>str</b>     |
| Anchor used to resolve {at}. |                |
| Argument                     |                |
| {side}: "north-east"         | <b>str</b>     |

Preset label side. Supported values are "east", "west", "north", "south", and diagonal combinations.

Argument

{label-at}: **auto**

**auto** | dictionary

Explicit CeTZ coordinate for the label.

Argument

{leader}: **"curve"**

**str**

Leader style: "curve", "straight", or "elbow".

Argument

{leader-end}: **auto**

**auto** | dictionary

Manual CeTZ coordinate where the leader should stop.

Argument

{target-gap}: **0.2**

**float**

Clearance from the target when {leader-end} is automatic.

label-offset and target-offset adjust automatic placement. leader-start, leader-end, leader-points, leader-start-offset, and leader-end-offset provide exact routing. mark and stroke style the line. label-size, label-gap, label-inset, label-radius, label-fill, label-stroke, and boxed style the label; name assigns an optional CeTZ object name.

```
#arrow-annotation(
  {from},
  {to},
  {label}: none,
  {from-anchor}: "mid",
  {to-anchor}: "mid",
  {label-anchor}: "south",
  {label-offset}: (0, 0.45),
  {mark}: (end: ">>", fill: luma(0%)),
  {stroke}: luma(0%),
  {boxed}: false,
  {label-size}: 0.85em,
  {label-fill}: luma(100%),
  {label-stroke}: 0.35pt + luma(72%),
  {label-inset}: 2pt,
  {label-radius}: 2pt,
  {name}: none
) → annotation
```

Build a free arrow overlay. from-anchor and to-anchor resolve named endpoints; label-anchor and label-offset position the optional label. mark and stroke style the arrow. boxed,

label-size, label-fill, label-stroke, label-inset, and label-radius style its label; name assigns an optional CeTZ object name.

```
#Label-annotation(  
  {at},  
  {label},  
  {anchor}: "mid",  
  {label-anchor}: "south",  
  {offset}: (0, 0.45),  
  {boxed}: false,  
  {label-size}: 0.85em,  
  {label-fill}: luma(100%),  
  {label-stroke}: 0.35pt + luma(72%),  
  {label-inset}: 2pt,  
  {label-radius}: 2pt,  
  {name}: none  
) →
```

**annotation**

Build a free text label overlay without a leader line. anchor, label-anchor, and offset control placement. boxed, label-size, label-fill, label-stroke, label-inset, and label-radius style the label; name assigns an optional CeTZ object name.

```
#cetz-annotation({body}, {name}: none) → annotation
```

Run custom CeTZ code after the molecule is drawn.

Argument

{body}

function

Function receiving the generated molecule name.

Argument

{name}: none

str | none

Optional name retained in the annotation descriptor.

## Part XIII

# Limitations

- Reaction SMILES/RXN input, atom mapping, automatic reaction-center detection/highlighting, and MOL2 input are intentionally deferred to a later release. Reaction schemes in this manual are manual Typst/CeTZ composition of separate molecule renderings.
- User-defined and unknown internal COLLECTIONs have no standard default glyph and remain inspection-only. MDLV30/HILITE currently depicts atom and bond members; SGroup, 3D-object, R-group, and generic members are preserved and reported by strict fidelity.
- Unknown future SGroup type codes are preserved for inspection but are not given an invented bracket convention; strict fidelity reports them. Multi-atom SUP groups without a usable label or with overlapping membership are preserved rather than contracted.
- M AP0 and ATTCHPT are R-group-member properties. Main-CTAB occurrences are retained for inspection but rejected by strict fidelity; full nested R-group member rendering is not yet implemented.
- Full mode can become crowded for large molecules because explicit hydrogens and atom labels occupy real page space. Prefer abbreviated or skeletal mode, or increase `{atom-sep}`.
- A valid Molfile/SDF 2D layout is preserved even when its full-mode labels are crowded. Automatic relay layout is limited to collapsed or numerically unstable XY coordinates.
- SMILES and unusable SDF coordinates are laid out with Coordgen. The result is deterministic for a given bundled plugin, but it may differ from an external chemical drawing program.
- Layout preserves explicit SDF wedge, parity, and enhanced-stereo metadata. It does not infer stereochemistry solely from 3D coordinates.
- Extended-chirality layout rotates ligand branches around a stereocenter. Invalid or cyclic topology may therefore fall back to a textual chirality annotation when its branches cannot be moved independently.
- Annotation helpers cover common callouts and arrows, not automatic collision-free figure composition. For complex layouts, use `#cetz-annotation` or dump the generated alchemist code.
- Rendering CI catches compilation failures and package/CLI source divergence on the listed Typst versions. The highlight corpus additionally checks atom/bond membership, SVG subpath topology, circle-versus-rounded-rectangle selection, and non-zero compound fills. This is not a general pixel-snapshot guarantee, so review fonts and final PDF appearance for publication output.
- Maintainers can run `scripts/check-pubchem-visual-regression.py` with the ignored local PubChem corpus for opt-in pixel regression checks; its baseline is machine-local and is not distributed with the package.

## Part XIV

# License and Dependencies

The molchemist-authored Typst source is distributed under the MIT License. The published package also embeds precompiled WASM components, so its manifest uses the aggregate SPDX expression MIT AND BSD-3-Clause AND Apache-2.0 AND (Apache-2.0 WITH LLVM-exception). Molfile / SDF parsing is powered by `sdfrust`; SMILES parsing is based on `opensmiles`; SMILES 2D coordinate generation uses `CoordgenLibs`; rendering is handled by `alchemist` and `CeTZ`. See the distributed third-party notices for the complete file-to-license mapping and license texts.

The example SDF files and rendered example images are attributed separately from the package code. In particular, this manual includes PubChem-derived example structures such as [CID 241](#). See `THIRD_PARTY_NOTICES.md` and `docs/assets/README.md` for source URLs and the relevant NCBI data-usage policy.

# Part XV

## Index

### A

|                                      |    |
|--------------------------------------|----|
| <code>anchor</code> .....            | 32 |
| <code>annotation</code> .....        | 32 |
| <code>#arrow-annotation</code> ..... | 37 |
| <code>#atom-anchor</code> .....      | 35 |
| <code>#atom-ref</code> .....         | 35 |

### B

|                                 |    |
|---------------------------------|----|
| <code>#bond-anchor</code> ..... | 35 |
| <code>#bond-ref</code> .....    | 35 |

### C

|                                       |    |
|---------------------------------------|----|
| <code>#callout-annotation</code> .... | 36 |
| <code>#cetz-annotation</code> .....   | 38 |

### I

|                                 |    |
|---------------------------------|----|
| <code>#inspect-mol</code> ..... | 33 |
|---------------------------------|----|

### L

|                                      |    |
|--------------------------------------|----|
| <code>#label-annotation</code> ..... | 38 |
|--------------------------------------|----|

### M

|                                     |    |
|-------------------------------------|----|
| <code>#molecule-anchor</code> ..... | 35 |
|-------------------------------------|----|

### R

|                                   |    |
|-----------------------------------|----|
| <code>#render-mol</code> .....    | 32 |
| <code>#render-smiles</code> ..... | 34 |